

Data Structures

Lecture 3

Algorithms II

References:

Introduction to Algorithms, Data Structures and Formal Languages.

Dinneen, M.J., Gimel'farb, G.L. and Wilson, M.C., 2009.



University of Kufa
Faculty of Computer Science
and Mathematics

Dr. Munqath Al-Attar

Information Technology Research and
Development Center ITRDC

Email:

munqith.alattar@uokufa.edu.iq

Website:

<http://staff.uokufa.edu.iq/profile.html?munqith.alattar>

Topics

- Algorithms
 - Loops Running time
 - “Big-Oh” tool
 - Algorithms complexity analysis

Algorithms II

Loops Running time

- The nested loops result in running time $T(n) = cn^k$ where k is the highest level of nesting (If the number of elementary operations in the **innermost loop** is constant).
- The first three values of k have special names:
 - **Linear time** for $k = 1$ (a single loop),
 - **Quadratic time** for $k = 2$ (two nested loops),
 - **Cubic time** for $k = 3$ (three nested loops).



Algorithms II

Loops Running time

What is the runtime of $g(n)$, if $f()$ is only one elementary operation?

```
void g(int n)
{
    for (int i = 0; i < n; ++i)
        f();
}
```

n

```
void g(int n)
{
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            f();
}
```

n^2

```
void g(int n)
{
    for (int i = 0; i < n; ++i)
        for (int j = 0; j <= i; ++j)
            f();
}
```

$$1 + 2 + 3 + \dots + n \approx \frac{n^2 + n}{2}$$

Algorithms II

“Big-Oh” tool

Big O

It is used to measure the **worst-case** scenarios of algorithms.

(Typically, the smallest complexity is used).

Examples

T(n)	Complexity
$5n^3 + 200n^2 + 15$	$O(n^3)$
$3n^2 + 2^{300}$	$O(n^2)$
$2 \log n^3$	$O(\log n)$
$4n + \log n$	$O(n)$
2^{46}	$O(1)$

Algorithms II

“Big-Oh” tool

Complexity ranking

Function	Complexity Title
n^2	Quadratic
n	Linear
$\text{Log } n$	logarithmic
1	Constant
$n!$	Factorial
2^n	Exponential
$N^d, d>3$	Polynomial

Algorithms II

Algorithms complexity analysis

Let us assume that $f()$ has complexity $O(1)$

```
for (int i = 0; i < n; ++i)
    f();
```

```
for (int i = 0; i < n; ++i)
    for (int j = 0; j < n; ++j)
        f();
```

```
for (int i = 0; i < n; ++i)
    for (int j = 0; j < n; ++j)
        for (int k = 0; k < n; ++k)
            f();
```

```
for (int i = 0; i < m; ++i)
    for (int j = 0; j < n; ++j)
        for (int k = 0; k < p; ++k)
            f();
```

```
if (condition)
{   O(n)   }
Else
{   O(n2)  }
```

Algorithms II

Algorithms complexity analysis

Let us assume that $f()$ has complexity $O(1)$

```
for (int i = 0; i < n; ++i)
    f();
```

$O(n)$

```
for (int i = 0; i < n; ++i)
    for (int j = 0; j < n; ++j)
        f();
```

$O(n^2)$

```
for (int i = 0; i < n; ++i)
    for (int j = 0; j < n; ++j)
        for (int k = 0; k < n; ++k)
            f();
```

$O(n^3)$

```
for (int i = 0; i < m; ++i)
    for (int j = 0; j < n; ++j)
        for (int k = 0; k < p; ++k)
            f();
```

$O(mnp)$

```
if (condition)
{
    O(n)
}
Else
{
    O(n^2)
}
```

$O(n^2)$

Algorithms II

Algorithms complexity analysis

Exercise II:

```
Void f(int n)    {  
    If (n > 0) {  
        DoSomething(n); // O(n)  
        f(n/2);  
    }  
}
```

$$T(n) = n + T(n/2)$$

$$T(n) = n + n/2 + n/4 + n/8 + \dots + 2 + 1$$

$$2 * T(n) = 2n + n + n/2 + n/4 + n/8 + \dots + 4 + 2$$

$$2 * T(n) - T(n) = T(n) = 2n - 1$$

$$T(n) \text{ is } O(n)$$

Java Function:

